

VINGO

End-to-End User Journey — Complete Flow Guide

This guide walks through the complete journey of Vingo — from an owner adding a restaurant, to a user discovering it, placing an order, and receiving it at their door. Every step is explained with the exact data flows, API calls, socket events, and backend logic involved.



Phase 1 Restaurant Owner Sets Up

Before any user can order, a restaurant owner must register, create their restaurant profile, get approved by the admin, and publish their menu. Here is exactly what happens at each step.

Step-by-step breakdown

1	Owner registers an account <i>role = 'owner' · POST /api/auth/register</i> Owner signs up with email and password. bcrypt(12) hashes the password. JWT access token (15 min) returned immediately. Role field is set to 'owner' in the Users collection. The owner can also use Google OAuth — account is auto-linked on email match.
2	Creates restaurant profile <i>POST /api/restaurants</i> Owner submits restaurant name, cuisine tags, address, cover photo, FSSAI cert, and opening hours. Address is geocoded to GeoJSON coordinates (lng, lat) and stored with a 2dsphere index for proximity search. isApproved defaults to false — restaurant is invisible to users at this stage. Cover photo uploaded to Cloudinary; URL stored in the document.
3	Admin reviews and approves <i>PATCH /api/restaurants/:id · Admin sets isApproved = true</i> Super Admin sees the pending restaurant in the Admin Panel. Admin checks name, FSSAI cert, and location. On approval, isApproved flips to true in MongoDB. BullMQ worker sends a confirmation email to the owner. Restaurant is now indexable by the search and geo-query APIs.
4	Owner adds menu items <i>POST /api/restaurants/:id/menu/items</i> Owner creates categories (e.g. Starters, Mains, Drinks) and adds items within them. Each item has: name, description, price (in paise), photo (Cloudinary), veg/non-veg flag, customisation options (size, spice, add-ons), and availability windows. Items outside their time window are hidden automatically.
5	Owner publishes menu <i>POST /api/restaurants/:id/menu/publish</i> Publishing creates a versioned snapshot of the menu in MongoDB. Older versions are retained so that any in-flight orders reference the price that was correct at order time, not a later edited price. Redis cache for this restaurant's menu is invalidated immediately.
6	Owner sets opening hours and goes live <i>PATCH /api/restaurants/:id/status</i> Owner configures Mon–Sun open/close times and toggles isOpen = true. Restaurant now appears in user search results, geo-filtered listings, and cuisine browsing. A cron job auto-resets isOpen based on schedule if the owner forgets to toggle.

What is stored in MongoDB after setup

Collection	Restaurants
Key fields	name, cuisines, location (GeoJSON), ownerId, isApproved, isOpen, openingHours, avgRating, coverImage
Menu stored	Separate Menus collection — restaurantId ref, categories[], items[] with versioned publish

Index used

2dsphere on location (for proximity search), text index on name + cuisines (full-text search)

From the user's perspective: as soon as isApproved = true and isOpen = true, the restaurant appears in search results within seconds — Redis cache TTL for listings is 10 minutes, but the geo-query bypasses cache and hits MongoDB directly for accuracy.

Phase 2 User Discovers and Places an Order

The user opens the Vingo app, finds the restaurant, builds a cart, pays, and the order lands on the restaurant's dashboard in real time. Here is every step in detail.

Step-by-step breakdown

1	User searches and finds the restaurant <i>GET /api/restaurants?lat=&lng;=&cuisine;=&rating;=</i> User's location is sent with the request. MongoDB \$nearSphere query finds open restaurants within 5 km, sorted by composite score (distance + rating + delivery time). Full-text search on name and cuisine uses MongoDB Atlas Search (Lucene) with fuzzy matching. Results are cached in Redis for 10 minutes per city+filter combination.
2	User views the restaurant and menu <i>GET /api/restaurants/:id</i> Returns full profile: cover photo (Cloudinary CDN), rating breakdown, offer badges, active menu snapshot (current published version), item availability (time-window filtered). Response cached in Redis with 5-minute TTL. AI recommendation engine also returns 'Picked For You' items based on the user's taste vector.
3	User adds items to cart <i>Stored in Redis — key: cart:{userId}</i> Cart is not saved to MongoDB yet — it lives in Redis with a 2-hour TTL. Each cart entry stores: itemId, restaurantId, quantity, selected customisations, unit price snapshot. If the user switches devices, cart syncs via userId key. On adding items from a different restaurant, user is warned and cart is cleared on confirmation.
4	User goes to checkout <i>Cart validated server-side on checkout attempt</i> Server re-fetches each item from MongoDB to verify it is still available and price hasn't changed. User selects delivery address (saved addresses or new via Google Places Autocomplete). Coupon code validated: type, min-order threshold, per-user limit, expiry all checked server-side — never trusted from client. GST (CGST + SGST) computed server-side. Delivery fee = base + per-km rate × surge multiplier (from Delivery Intelligence Engine).
5	User pays via Razorpay <i>POST /api/orders → Razorpay Order created server-side</i> Server creates a Razorpay Order with the final amount (in paise). An idempotency key is stored in Redis so a duplicate request within 10 min returns the same order instead of charging twice. Razorpay Checkout SDK opens on the client. Supports: UPI, Net Banking, Cards, Paytm/PhonePe, COD (with fraud-score threshold).
6	Payment verified server-side <i>POST /api/payments/verify · HMAC-SHA256 signature check</i> After Razorpay reports success, client sends paymentId + orderId + signature to the server. Server recomputes HMAC-SHA256(orderId + ' ' + paymentId) using the Razorpay secret key and compares to the received signature. Mismatch = 400 error, payment not credited. Match = order status updated to PAYMENT_CONFIRMED in MongoDB.

7

Order saved and restaurant notified

Order written to MongoDB · Socket.IO fires order_placed

Order document created with full item snapshot (prices at time of order), status timeline (append-only array), delivery address snapshot, and OTP (hashed). Cart deleted from Redis. Loyalty points and wallet cashback previews queued. Socket.IO emits order_placed to room restaurant:{restaurantId} — restaurant owner sees the new order appear on their Kanban dashboard instantly. BullMQ sends confirmation email and push notification (FCM) to the user.

Payment data flow summary

Razorpay flow	Client → server creates Order → SDK opens → user pays → client gets paymentId + signature → server verifies HMAC → order confirmed
Idempotency	Redis key: idempotency:{userId}:{cartHash} · TTL 10 min · prevents double-charge on retry
Webhook	Razorpay also fires async webhook to POST /api/payments/webhook — server re-verifies and reconciles in case client callback was lost
Wallet payment	If user selects wallet: MongoDB transaction atomically debits walletBalance and creates transaction record — no Razorpay involved
COD	Max order value guard applied; delivery partner receives COD flag; payment marked on DELIVERED status

Security note: the server NEVER trusts the amount from the client. The final amount is always computed server-side from the cart snapshot + coupon validation + tax computation + delivery fee. The Razorpay Order is created with this server-computed amount.

Phase 3 Fulfillment — Restaurant to Doorstep

Once the order is confirmed, the restaurant accepts it, the delivery engine assigns a partner, food is prepared, picked up, and delivered. Every status change is pushed to all parties in real time via Socket.IO.

Step-by-step breakdown

1	Restaurant receives order on dashboard <i>Socket.IO room: restaurant:{restaurantId} · event: order_placed</i> The owner's Kanban board has three columns: New / Preparing / Ready. The new order card slides into the New column in real time — no page refresh needed. Card shows: items, quantities, customisations, delivery address distance, order total, and a countdown. If the owner doesn't act within 5 minutes, an auto-accept job fires from BullMQ.
2	Owner accepts the order <i>PATCH /api/orders/:id/accept · status → ACCEPTED</i> Owner taps Accept and enters estimated prep time (default 15 min). Status updated to ACCEPTED in MongoDB; statusTimeline array gets a new entry with timestamp. Delivery Intelligence Engine is triggered immediately to start scoring available partners. User receives push notification: 'Your order has been accepted!'.
3	Delivery partner assignment <i>Delivery engine scores partners · Redis lock on assignment</i> MongoDB \$nearSphere query finds all partners with status='available' within 3 km. Each candidate scored: distance (40%) + rating (30%) + acceptance rate (20%) + shift time remaining (10%). Top-scored partner gets a Socket.IO offer (delivery_assigned event). Partner has 30 seconds to accept in their app. If declined or timeout: next candidate tried. Max 3 attempts before admin alert fires. On acceptance: Redis key idempotency:assign:{orderId} written to prevent race condition across multiple Node.js instances assigning the same order twice.
4	Food is prepared <i>Owner updates status · PATCH /api/orders/:id status → PREPARING</i> Order card moves to Preparing column on the Kanban. User's tracking screen shows 'Chef is preparing your food' with a pulsing animation. Owner can print a thermal ticket via the browser print API for kitchen use.
5	Owner marks food ready <i>PATCH /api/orders/:id/ready · status → READY</i> Order moves to the Ready column. Delivery partner app shows a notification: 'Order is ready for pickup!' Mapbox route displayed in partner's app from their current location to the restaurant. ETA to restaurant computed by Mapbox Directions API + traffic data.
6	Delivery partner picks up the order <i>PATCH /api/orders/:id/pickup · status → OUT_FOR_DELIVERY</i> Partner arrives at restaurant, collects the order, and taps 'Picked Up' in the app. This API call: updates order status, records timestamp in statusTimeline, emits order_update Socket.IO event to room order:{orderId} (user + owner both notified). Partner's GPS ping interval starts: device emits gps_ping event every 3 seconds via Socket.IO.

7

Real-time GPS tracking*Socket.IO event: gps_ping · stored in DeliveryTracking collection*

Each gps_ping payload: { lat, lng, heading, speed, accuracy, timestamp }. Server writes to DeliveryTracking (TTL index: auto-deleted after 72h). A Kalman filter approximation smooths jitter before broadcasting location_update to the user. ETA recalculated every 90 seconds via Mapbox Directions API + partner's real speed. Updated ETA pushed via eta_update event. User sees a live polyline on the map.

8

OTP delivery confirmation*Socket.IO event: otp_request · POST /api/orders/:id/verify-otp*

When partner's GPS location comes within 50 m of the delivery address, server generates a 6-digit OTP and pushes it to the partner's app via Socket.IO. User is prompted to share the OTP with the delivery partner. Partner enters OTP in app → POST /api/orders/:id/verify-otp. Server verifies the hash → status → DELIVERED.

9

Order delivered — post-delivery triggers*status → DELIVERED · BullMQ fires multiple jobs*

1. Loyalty points credited: 1 point per Rs10 spent (double on partner restaurants weekends). 2. Wallet cashback credited if applicable (BullMQ job, async). 3. Review prompt scheduled: FCM push notification sent 15 minutes after delivery. 4. Invoice PDF generated (Puppeteer-rendered) and emailed to user. 5. Owner revenue analytics updated via aggregation pipeline. 6. Partner earnings record appended to their earnings ledger.

Phase 0 Deep Dive — How the Delivery Partner Picks Up

This is the most technically interesting part of the flow. Here is exactly what happens from the moment the restaurant marks food ready to the moment the partner taps 'Picked Up'.

The assignment algorithm in detail

- Server runs: `db.users.find({ role:'delivery', status:'available', location: { $nearSphere: { $geometry: restaurantCoords, $maxDistance: 3000 } } })`
- Each result gets a composite score: $\text{score} = (1/\text{distanceKm}) * 0.4 + \text{rating} * 0.3 + \text{acceptanceRate} * 0.2 + (\text{shiftRemaining}/8) * 0.1$
- Results sorted descending by score. Top partner gets the offer first.
- Offer delivered via Socket.IO to that partner's connected device only (individual socket room).
- 30-second acceptance window enforced via a BullMQ delayed job — if the job fires, partner is skipped.
- On acceptance: `SET assign:{orderId} {partnerId} EX 86400 NX` in Redis — NX flag ensures only one assignment ever wins.
- If all 3 candidates decline: admin Slack alert + order flagged for manual intervention.

Socket.IO events during pickup

Event	Direction	Payload
delivery_assigned	Server → Partner app	orderId, restaurantName, address, items summary, estimated earnings
partner_en_route	Server → User + Owner	Partner accepted; name, photo, vehicle, phone hash, ETA to restaurant
order_update (READY)	Server → Partner app	Food is ready for collection — fires when owner taps Ready
gps_ping	Partner → Server	{ lat, lng, heading, speed, accuracy } — every 3 seconds after pickup
location_update	Server → User app	Smoothed location for map polyline — broadcast after Kalman filtering
eta_update	Server → User app	Revised ETA — recalculated every 90 seconds via Mapbox
otp_request	Server → Partner app	6-digit OTP when partner is within 50 m of delivery address
delivery_completed	Server → User + Owner	Order status DELIVERED — triggers review prompt, earnings, points

GPS tracking technical detail

- Partner device emits `gps_ping` via Socket.IO (WebSocket) every 3 seconds while status = `OUT_FOR_DELIVERY`.
- Server writes each ping to `DeliveryTracking` collection with TTL index (auto-delete after 72 hours).
- Kalman filter smoothing: server maintains a running state estimate per partner, blending new GPS reading with predicted position based on last known heading and speed. This removes GPS jitter from poor signal areas.
- Smoothed position broadcast to room `order:{orderId}` as `location_update` event — user's map polyline extends.
- ETA engine: every 90 seconds, server calls Mapbox Directions API with current partner position as origin and delivery address as destination. Response duration (accounting for live traffic) replaces the ETA shown to the user.

- User can share a live tracking link: POST /api/tracking/:orderId/share-link returns a JWT-signed URL valid for 4 hours — anyone with the link can see the map without logging in.

OTP confirmation detail

- Server computes distance between partner's latest GPS ping and the delivery address every ping.
- When distance drops below 50 metres: server generates a 6-digit random OTP, hashes it with bcrypt, stores hash on the Order document.
- Plain OTP pushed to partner app via Socket.IO (otp_request event) — never sent to the user over the network.
- User is shown a prompt: 'Share this code with your delivery partner'. Code displayed in large text.
- Partner enters code in app → POST /api/orders/:id/verify-otp. Server bcrypt-compares entered code with stored hash.
- Match: status → DELIVERED, timestamp recorded, all post-delivery BullMQ jobs enqueued.
- Wrong code: 400 error returned. Three wrong attempts: partner flagged for investigation, order escalated.
- OTP expires after 30 minutes regardless — if not confirmed, admin is alerted.

Why OTP instead of just GPS? GPS confirms the partner was near the address but not that they actually handed over the food. OTP confirmation requires a physical exchange — the user must verbally share the code. This prevents 'fake delivery' fraud where a partner marks an order delivered without attempting delivery.

Order state machine — full reference

PLACED	Order created after cart validation but before payment	User, BullMQ
PAYMENT_PENDING	Awaiting Razorpay payment completion	Razorpay SDK
PAYMENT_CONFIRMED	HMAC signature verified; funds captured	Server webhook
ACCEPTED	Restaurant confirmed order; prep time set	Owner
PREPARING	Kitchen is preparing the food	Owner
READY	Food packed and ready for pickup	Owner
PICKED_UP	Delivery partner has collected the order	Partner
OUT_FOR_DELIVERY	Partner en route to delivery address; GPS active	Partner
DELIVERED	OTP verified; order completed	Partner + Server
CANCELLED	Cancelled within window; refund initiated	User / Auto
REFUND_INITIATED	Refund queued in BullMQ; 5–7 days to source	Server

Every status change appends an entry to the order's `statusTimeline` array: `{ status, timestamp, actorId }`. This array is append-only — no updates, no deletes. It forms a full audit trail of when each transition happened and who triggered it.

Vingo end-to-end: Owner setup · User discovery · Cart + payment · Real-time assignment · GPS tracking · OTP delivery · Post-delivery rewards